Knowledge Engineering: The Seven-Step Process

An Extended Example in Electronic Circuits

Stuart J. Russell and Peter Norvig
Compiled by
Kiran Bagale

December, 2025

1 The Knowledge Engineering Process

Knowledge engineering projects vary widely in content, scope, and difficulty, but all such projects include the following steps:

1.1 Step 1: Identify the Task

The knowledge engineer must delineate the range of questions that the knowledge base will support and the kinds of facts that will be available for each specific problem instance. For example:

- Does the wumpus knowledge base need to be able to choose actions or only answer questions about the environment's contents?
- Will the sensor facts include the current location?

The task will determine what knowledge must be represented in order to connect problem instances to answers. This step is analogous to the PEAS process for designing agents.

1.2 Step 2: Assemble the Relevant Knowledge

The knowledge engineer might already be an expert in the domain, or might need to work with real experts to extract what they know—a process called **knowledge acquisition**. At this stage, the knowledge is not represented formally. The idea is to:

- Understand the scope of the knowledge base, as determined by the task
- Understand how the domain actually works

For the wumpus world, which is defined by an artificial set of rules, the relevant knowledge is easy to identify. However, notice that the definition of adjacency was not supplied explicitly in the wumpus-world rules. For real domains, the issue of relevance can be quite difficult—for example, a system for simulating VLSI designs might or might not need to take into account stray capacitances and skin effects.

1.3 Step 3: Decide on a Vocabulary

Translate the important domain-level concepts into logic-level names. This involves:

- Choosing predicates, functions, and constants
- Making knowledge-engineering style decisions (similar to programming style)

- Questions to consider:
  - Should pits be represented by objects or by a unary predicate on squares?
  - Should the agent's orientation be a function or a predicate?
  - Should the wumpus's location depend on time?

Once the choices have been made, the result is a vocabulary that is known as the **ontology** of the domain. The word *ontology* means a particular theory of the nature of being or existence. The ontology determines what kinds of things exist, but does not determine their specific properties and interrelationships.

1.4 Step 4: Encode General Knowledge About the Domain

The knowledge engineer writes down the axioms for all the vocabulary terms. This pins down (to the extent possible) the meaning of the terms, enabling the expert to check the content. Often, this step reveals misconceptions or gaps in the vocabulary that must be fixed by returning to step 3 and iterating through the process.

1.5 Step 5: Encode a Description of the Specific Problem Instance

If the ontology is well thought out, this step will be easy. It will involve writing simple atomic sentences about instances of concepts that are already part of the ontology. For a logical agent, problem instances are supplied by the sensors, whereas a "disembodied" knowledge base is supplied with additional sentences in the same way that traditional programs are supplied with input data.

1.6 Step 6: Pose Queries to the Inference Procedure

This is where the reward is: we can let the inference procedure operate on the axioms and problem-specific facts to derive the facts we are interested in knowing. Thus, we avoid the need for writing an application-specific solution algorithm.

1.7 Step 7: Debug the Knowledge Base

Alas, the answers to queries will seldom be correct on the first try. More precisely, the answers will be correct for the knowledge base as written, assuming that the inference procedure is sound, but they will not be the ones that the user is expecting.

1.7.1 Common Debugging Issues

Missing or Weak Axioms Missing axioms or axioms that are too weak can be easily identified by noticing places where the chain of reasoning stops unexpectedly. For example, if the knowledge base includes a diagnostic rule for finding the wumpus:

$$\forall s \; Smelly(s) \Rightarrow Adjacent(Home(Wumpus), s)$$

instead of the biconditional, then the agent will never be able to prove the absence of wumpuses.

Incorrect Axioms Incorrect axioms can be identified because they are false statements about the world. For example:

$$\forall x \; NumOfLegs(x, 4) \Rightarrow Mammal(x)$$

is false for reptiles, amphibians, and, more importantly, tables. The falsehood of this sentence can be determined independently of the rest of the knowledge base.

In contrast, a typical error in a program looks like:

```
offset = position + 1
```

It is impossible to tell whether this statement is correct without looking at the rest of the program to see whether, for example, `offset` is used to refer to the current position, or to one beyond the current position, or whether the value of `position` is changed by another statement.

2 The Electronic Circuits Domain

We will develop an ontology and knowledge base that allow us to reason about digital circuits. We follow the seven-step process for knowledge engineering.

2.1 Step 1: Identify the Task

There are many reasoning tasks associated with digital circuits. At the highest level, one analyzes the circuit's **functionality**:

- Does the circuit actually add properly?
- If all the inputs are high, what is the output of gate A2?

Questions about the circuit's **structure** are also interesting:

- What are all the gates connected to the first input terminal?
- Does the circuit contain feedback loops?

These will be our tasks in this section. There are more detailed levels of analysis, including those related to timing delays, circuit area, power consumption, production cost, and so on. Each of these levels would require additional knowledge.

2.2 Step 2: Assemble the Relevant Knowledge

What do we know about digital circuits? For our purposes:

- They are composed of **wires** and **gates**
- Signals flow along wires to the input terminals of gates
- Each gate produces a signal on the output terminal that flows along another wire

To determine what these signals will be, we need to know how the gates transform their input signals:

- **AND, OR, and XOR gates** have two input terminals
- **NOT gates** have one input terminal
- All gates have one output terminal
- Circuits, like gates, have input and output terminals

Important Observation To reason about functionality and connectivity, we do not need to talk about the wires themselves, the paths they take, or the junctions where they come together. All that matters is the connections between terminals—we can say that one output terminal is connected to another input terminal without having to say what actually connects them. Other factors such as the size, shape, color, or cost of the various components are irrelevant to our analysis.

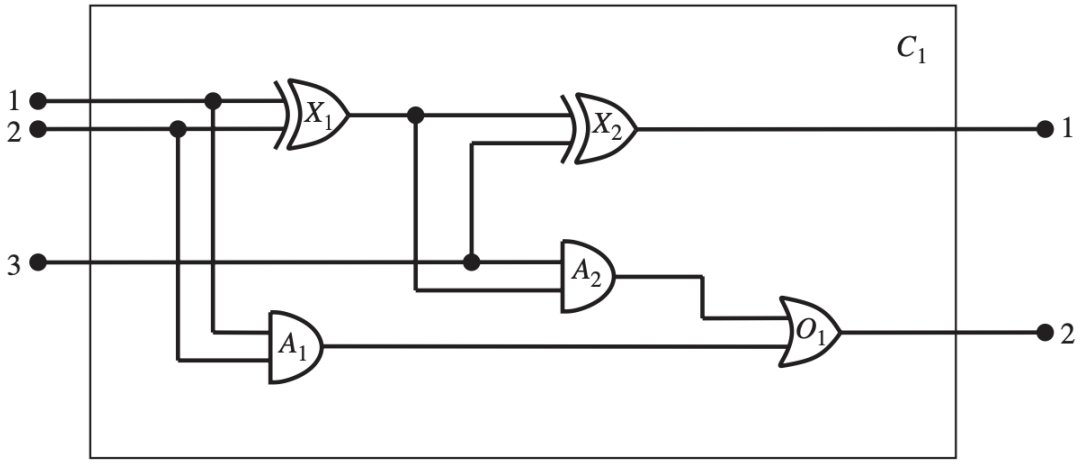

Figure 1: A digital circuit C1, purporting to be a one-bit full adder. The first two inputs are the two bits to be added, and the third input is a carry bit. The first output is the sum, and the second output is a carry bit for the next adder. The circuit contains two XOR gates, two AND gates, and one OR gate.

Alternative Ontologies If our purpose were something other than verifying designs at the gate level, the ontology would be different:

- For **debugging faulty circuits**: include the wires in the ontology, because a faulty wire can corrupt the signal
- For **resolving timing faults**: include gate delays
- For **designing a profitable product**: include cost and speed relative to competitors

2.3 Step 3: Decide on a Vocabulary

We now know that we want to talk about circuits, terminals, signals, and gates. The next step is to choose functions, predicates, and constants to represent them.

2.3.1 Gates

- Each gate is represented as an object named by a constant
- Predicate: $\mathit{Gate}(X_1)$ asserts that X_1 is a gate
- Function: $\texttt{Type}(X_1) = \texttt{XOR}$ specifies the gate type
- Gate types are constants: `AND`, `OR`, `XOR`, `NOT`

2.3.2 Circuits

- Predicate: $\mathit{Circuit}(C_1)$ identifies circuits

2.3.3 Terminals

- Predicate: $\mathit{Terminal}(x)$ identifies terminals
- Function: $\texttt{In}(1, X_1)$ denotes the first input terminal for gate X_1
- Function: $\texttt{Out}(1, X_1)$ denotes the first output terminal for gate X_1
- Function: $\mathit{Arity}(c, i, j)$ says that circuit c has i input and j output terminals

2.3.4 Connectivity

- Predicate: $\mathit{Connected}(t_1, t_2)$ indicates that terminals t_1 and t_2 are connected
- Example: $\mathit{Connected}(\texttt{Out}(1, X_1), \texttt{In}(1, X_2))$

2.3.5 Signals

- Objects: signal values 1 and 0
- Function: $\texttt{Signal}(t)$ denotes the signal value for terminal t
- This allows queries like: "What are all the possible values of the signals at the output terminals of circuit C_1?"

2.4 Step 4: Encode General Knowledge of the Domain

One sign that we have a good ontology is that we require only a few general rules, which can be stated clearly and concisely. These are all the axioms we will need:

Axiom 1: Connected Terminals If two terminals are connected, then they have the same signal:

$$\forall t_1, t_2 \ \mathit{Terminal}(t_1) \wedge \mathit{Terminal}(t_2) \wedge \mathit{Connected}(t_1, t_2) \Rightarrow \texttt{Signal}(t_1) = \texttt{Signal}(t_2)$$

Axiom 2: Binary Signals The signal at every terminal is either 1 or 0:

$$\forall t \ \mathit{Terminal}(t) \Rightarrow \texttt{Signal}(t) = 1 \vee \texttt{Signal}(t) = 0$$

Axiom 3: Connectivity is Commutative

$$\forall t_1, t_2 \ \mathit{Connected}(t_1, t_2) \Leftrightarrow \mathit{Connected}(t_2, t_1)$$

Axiom 4: Gate Types There are four types of gates:

$$\forall g \ \mathit{Gate}(g) \wedge k = \texttt{Type}(g) \Rightarrow k = \texttt{AND} \vee k = \texttt{OR} \vee k = \texttt{XOR} \vee k = \texttt{NOT}$$

Axiom 5: AND Gate Behavior An AND gate's output is 0 if and only if any of its inputs is 0:

$$\forall g \ \mathit{Gate}(g) \wedge \texttt{Type}(g) = \texttt{AND} \Rightarrow \texttt{Signal}(\texttt{Out}(1, g)) = 0 \Leftrightarrow \exists n \ \texttt{Signal}(\texttt{In}(n, g)) = 0$$

Axiom 6: OR Gate Behavior An OR gate's output is 1 if and only if any of its inputs is 1:

$$\forall g \ \mathit{Gate}(g) \wedge \texttt{Type}(g) = \texttt{OR} \Rightarrow \texttt{Signal}(\texttt{Out}(1, g)) = 1 \Leftrightarrow \exists n \ \texttt{Signal}(\texttt{In}(n, g)) = 1$$

Axiom 7: XOR Gate Behavior An XOR gate's output is 1 if and only if its inputs are different:

$$\forall g \ \mathit{Gate}(g) \wedge \texttt{Type}(g) = \texttt{XOR} \Rightarrow \texttt{Signal}(\texttt{Out}(1, g)) = 1 \Leftrightarrow \texttt{Signal}(\texttt{In}(1, g)) \neq \texttt{Signal}(\texttt{In}(2, g))$$

Axiom 8: NOT Gate Behavior A NOT gate's output is different from its input:

$$\forall g \ \mathit{Gate}(g) \wedge \texttt{Type}(g) = \texttt{NOT} \Rightarrow \texttt{Signal}(\texttt{Out}(1, g)) \neq \texttt{Signal}(\texttt{In}(1, g))$$

Axiom 9: Gate Arity The gates (except for NOT) have two inputs and one output:

$$\forall g\ \mathit{Gate}(g) \wedge \texttt{Type}(g) = \texttt{NOT} \Rightarrow \mathit{Arity}(g, 1, 1)$$
$$\forall g\ \mathit{Gate}(g) \wedge k = \texttt{Type}(g) \wedge (k = \texttt{AND} \vee k = \texttt{OR} \vee k = \texttt{XOR}) \Rightarrow \mathit{Arity}(g, 2, 1)$$

Axiom 10: Circuit Terminals A circuit has terminals, up to its input and output arity, and nothing beyond its arity:

$$\forall c, i, j\ \mathit{Circuit}(c) \wedge \mathit{Arity}(c, i, j) \Rightarrow$$
$$\forall n\ (n \leq i \Rightarrow \mathit{Terminal}(\texttt{In}(c, n))) \wedge (n > i \Rightarrow \texttt{In}(c, n) = \texttt{Nothing}) \wedge$$
$$\forall n\ (n \leq j \Rightarrow \mathit{Terminal}(\texttt{Out}(c, n))) \wedge (n > j \Rightarrow \texttt{Out}(c, n) = \texttt{Nothing})$$

Axiom 11: Distinctness Gates, terminals, signals, gate types, and Nothing are all distinct:

$$\forall g, t\ \mathit{Gate}(g) \wedge \mathit{Terminal}(t) \Rightarrow g \neq t \neq 1 \neq 0 \neq \texttt{OR} \neq \texttt{AND} \neq \texttt{XOR} \neq \texttt{NOT} \neq \texttt{Nothing}$$

Axiom 12: Gates are Circuits

$$\forall g\ \mathit{Gate}(g) \Rightarrow \mathit{Circuit}(g)$$

2.5 Step 5: Encode the Specific Problem Instance

The circuit shown in Figure 8.6 (to be added) is encoded as circuit C_1 with the following description.

2.5.1 Circuit and Gate Categorization

$$\mathit{Circuit}(C_1) \wedge \mathit{Arity}(C_1, 3, 2)$$
$$\mathit{Gate}(X_1) \wedge \texttt{Type}(X_1) = \texttt{XOR}$$
$$\mathit{Gate}(X_2) \wedge \texttt{Type}(X_2) = \texttt{XOR}$$
$$\mathit{Gate}(A_1) \wedge \texttt{Type}(A_1) = \texttt{AND}$$
$$\mathit{Gate}(A_2) \wedge \texttt{Type}(A_2) = \texttt{AND}$$
$$\mathit{Gate}(O_1) \wedge \texttt{Type}(O_1) = \texttt{OR}$$

2.5.2 Connections Between Components

$$\mathit{Connected}(\texttt{In}(1, C_1), \texttt{In}(1, X_1))$$
$$\mathit{Connected}(\texttt{In}(1, C_1), \texttt{In}(1, A_1))$$
$$\mathit{Connected}(\texttt{In}(2, C_1), \texttt{In}(2, X_1))$$
$$\mathit{Connected}(\texttt{In}(2, C_1), \texttt{In}(2, A_1))$$
$$\mathit{Connected}(\texttt{In}(3, C_1), \texttt{In}(2, X_2))$$
$$\mathit{Connected}(\texttt{In}(3, C_1), \texttt{In}(1, A_2))$$
$$\mathit{Connected}(\texttt{Out}(1, X_1), \texttt{In}(1, X_2))$$
$$\mathit{Connected}(\texttt{Out}(1, X_1), \texttt{In}(2, A_2))$$
$$\mathit{Connected}(\texttt{Out}(1, A_2), \texttt{In}(1, O_1))$$
$$\mathit{Connected}(\texttt{Out}(1, A_1), \texttt{In}(2, O_1))$$
$$\mathit{Connected}(\texttt{Out}(1, X_2), \texttt{Out}(1, C_1))$$
$$\mathit{Connected}(\texttt{Out}(1, O_1), \texttt{Out}(2, C_1))$$

2.6 Step 6: Pose Queries to the Inference Procedure

2.6.1 Circuit Verification

Query 1: Specific Output Conditions What combinations of inputs would cause the first output of C_1 (the sum bit) to be 0 and the second output of C_1 (the carry bit) to be 1?

$$\exists i_1, i_2, i_3 \ \texttt{Signal}(\texttt{In}(1, C_1)) = i_1 \wedge \texttt{Signal}(\texttt{In}(2, C_1)) = i_2 \wedge \texttt{Signal}(\texttt{In}(3, C_1)) = i_3 \wedge \\ \texttt{Signal}(\texttt{Out}(1, C_1)) = 0 \wedge \texttt{Signal}(\texttt{Out}(2, C_1)) = 1$$

The answers are substitutions for the variables i_1, i_2, and i_3 such that the resulting sentence is entailed by the knowledge base. AskVars will give us three such substitutions:

$$\{i_1/1, i_2/1, i_3/0\}$$
$$\{i_1/1, i_2/0, i_3/1\}$$
$$\{i_1/0, i_2/1, i_3/1\}$$

Query 2: Complete Input-Output Table What are the possible sets of values of all the terminals for the adder circuit?

$$\exists i_1, i_2, i_3, o_1, o_2 \ \texttt{Signal}(\texttt{In}(1, C_1)) = i_1 \wedge \texttt{Signal}(\texttt{In}(2, C_1)) = i_2 \wedge \texttt{Signal}(\texttt{In}(3, C_1)) = i_3 \wedge \\ \texttt{Signal}(\texttt{Out}(1, C_1)) = o_1 \wedge \texttt{Signal}(\texttt{Out}(2, C_1)) = o_2$$

This final query will return a complete input-output table for the device, which can be used to check that it does in fact add its inputs correctly. This is a simple example of **circuit verification**. We can also use the definition of the circuit to build larger digital systems, for which the same kind of verification procedure can be carried out.

Many domains are amenable to the same kind of structured knowledge-base development, in which more complex concepts are defined on top of simpler concepts.

2.7 Step 7: Debug the Knowledge Base

We can perturb the knowledge base in various ways to see what kinds of erroneous behaviors emerge.

2.7.1 Example: Forgetting $1 \neq 0$

Suppose we fail to assert that $1 \neq 0$. Suddenly, the system will be unable to prove any outputs for the circuit, except for the input cases 000 and 110. We can pinpoint the problem by asking for the outputs of each gate.

Diagnostic Process For example, we can ask:

$$\exists i_1, i_2, o \ \texttt{Signal}(\texttt{In}(1, C_1)) = i_1 \wedge \texttt{Signal}(\texttt{In}(2, C_1)) = i_2 \wedge \texttt{Signal}(\texttt{Out}(1, X_1)) = o$$

which reveals that no outputs are known at X_1 for the input cases 10 and 01.

Then, we look at the axiom for XOR gates, as applied to X_1:

$$\texttt{Signal}(\texttt{Out}(1, X_1)) = 1 \Leftrightarrow \texttt{Signal}(\texttt{In}(1, X_1)) \neq \texttt{Signal}(\texttt{In}(2, X_1))$$

If the inputs are known to be, say, 1 and 0, then this reduces to:

$$\texttt{Signal}(\texttt{Out}(1, X_1)) = 1 \Leftrightarrow 1 \neq 0$$

Solution Now the problem is apparent: the system is unable to infer that $\texttt{Signal}(\texttt{Out}(1, X_1)) = 1$, so we need to tell it that $1 \neq 0$.

3 Conclusion

The seven-step knowledge engineering process provides a systematic approach to building knowledge bases:

1. Identify the task
2. Assemble the relevant knowledge
3. Decide on a vocabulary (ontology)
4. Encode general knowledge about the domain
5. Encode the specific problem instance
6. Pose queries to the inference procedure
7. Debug the knowledge base

This process is iterative, and steps 3–7 often need to be repeated as issues are discovered and the ontology is refined. The electronic circuits example demonstrates how a well-designed ontology can support complex reasoning with relatively few axioms.